BGMA: a budgeted memetic heuristic for the Euclidean TSP, audited against exact and published optima

Mohammadreza Aala
mrala@ut.ac.ir

2026-10-27

Abstract

We describe Beam-Guided Memetic Annealing (BGMA), a heuristic for the symmetric Euclidean travelling salesman problem, and evaluate it against known optima rather than lower bounds. BGMA combines standard components: a beam-search constructor, candidate-list 2-opt/Or-opt screening, an elite pool recombined by edge recombination, double-bridge and short-anneal perturbation, and a full 2-opt/Or-opt polish reserved for the best candidates. Five effort levels scale every budget with the instance size. Every level returns the Held–Karp optimum on all 26 instances with $n \leq 22$. On ten TSPLIB instances with $51 \leq n \leq 442$, the top level returns the published optimum in 89 of 100 seeded runs, but on none of the ten seeds for pcb442. On four instances with $574 \leq n \leq 1060$ its median gap is +0.50% to +2.10%, at 3.5 min to 58.4 min of CPU time per run. The decisive comparison is negative. An LK-style iterated local search from the same code base, restarted for the CPU time of one top-level BGMA run, matches BGMA's median tour on 9 of these 14 instances and beats it on the other 5, and on pr1002 it reaches BGMA's quality in 12.1 s instead of 39.8 min. We also show that the 1-tree bound is an unreliable yardstick: its distance below the optimum ranges over 0.00–5.34% across these instances, so an optimal fl417 tour reads +5.639% above the bound. BGMA is not competitive with LKH, EAX or Concorde, none of which we ran. The code, the harnesses and every data file behind the paper are released, and one script regenerates every number.

1 Introduction

The symmetric Euclidean travelling salesman problem (TSP) is NP-hard [25], and it has an unusually mature algorithmic toolbox. Branch-and-cut proves optimality for instances with tens of thousands of cities [1]. Lin–Kernighan-type local search [11, 12, 17] and edge-assembly genetic algorithms [22] find optimal or near-optimal tours for much larger ones. A new heuristic in this setting is only worth reporting if the report is precise about what was measured, against what reference, and at what cost.

This paper describes Beam-Guided Memetic Annealing (BGMA) and audits it in that spirit. BGMA combines standard parts: a beam-search constructor, candidate-list 2-opt/Or-opt, an elite pool recombined by edge recombination, double-bridge and short-anneal perturbation, and a full 2-opt/Or-opt polish that is applied only to the few candidates that can win. Five *effort levels* scale every budget with n and form a fixed-budget quality dial.

Contributions.

1. A complete, dependency-free and deterministic implementation with a five-level effort dial, and a precise description of what each level does (Section 3).

2. An evaluation against *known optima* rather than bounds or best-found tours: Held–Karp at $n \leq 22$, and the published optima of fourteen TSPLIB instances from $n = 51$ to 1060, with ten seeds up to $n = 442$ (Section 5).

3. A same-harness comparison with an LK-style iterated local search, including one at equal CPU time on TSPLIB. The result is negative for BGMA: restarted for the same time, the simpler search matches or beats BGMA's top level on every instance (Section 5.3).

4. Evidence that the Held–Karp 1-tree bound gap is unreliable as a quality measure for heuristics: its slack against the true optimum ranges from 0% to over 5% on TSPLIB, and is larger still on degenerate geometry (Sections 5.3 and 5.4).

Non-claims. BGMA carries no approximation guarantee. Its budgets are polynomial by construction, so its behaviour bears on nothing beyond its own empirical performance, and in particular not on P versus NP. We did not run Concorde, LKH or EAX, and we do not claim that BGMA is competitive with them; on the instances used here they are known to be stronger. The parameters were tuned on synthetic instances from the same generator used in Section 5.2.

The rest of the paper is organised as follows. Section 2 reviews related work, Section 3 describes the algorithm and Section 4 the protocol. Section 5 reports results, and Sections 6 and 7 discuss them and their limits.

2 Related work

Exact methods and bounds. The Bellman–Held–Karp dynamic program [3, 8] solves the TSP in $O(n^2 2^n)$ time and $O(n 2^n)$ memory. In practice that limits it to a little over twenty cities on a workstation, and that is how we use it: as ground truth at $n \leq 22$. The 1-tree relaxation with subgradient optimisation of node penalties [9, 10] gives a valid lower bound at polynomial cost. We use it as a certificate, and show that its slack depends strongly on the geometry. Branch-and-cut, as implemented in Concorde [1], routinely proves optimality for instances with thousands of cities, including every TSPLIB [26] instance used here.

Local search and its iterated forms. 2-opt [6], Or-opt [23] and Lin–Kernighan (LK) [17] are the classical neighbourhoods. Neighbour lists and don't-look bits [4] make them nearly linear in practice. Iterating a local search from double-bridge perturbations of the incumbent gives the large-step Markov chain or chained LK [19], a form of iterated local search [18]. Johnson and McGeoch survey and benchmark this family extensively [14, 15]. Helsgaun's LKH [11–13], with α-nearness candidate sets and 5-opt sequential moves, is the reference heuristic. On instances of the sizes studied here it usually finds the optimum in seconds.

Memetic and genetic algorithms. Memetic algorithms combine a population and recombination with local search [21]. Merz and Freisleben [20] give a thorough study for the TSP with LK as the local search. Edge recombination (ERX) [27] is one of the earliest edge-preserving crossovers. The edge assembly crossover (EAX) GA of Nagata and Kobayashi [22] is among the strongest TSP heuristics known and is often competitive with LKH on large instances.

Approximation and budgets. Christofides' algorithm [5] guarantees a factor of 3/2 on metric instances, and the Euclidean TSP admits a PTAS [2]. BGMA has no guarantee of either kind. Its

effort levels are *contract* budgets in the sense of Zilberstein [28]: the budget is fixed before the run, and the solver cannot be interrupted for a result.

Position of this work. Every component of BGMA is standard; the combination is ours. We do not claim that BGMA is competitive with LKH or EAX, and we did not run either. Instead the paper does three things. It measures BGMA against known optima rather than against bounds or best-found tours. It reports where the method's effort dial stops paying. And it compares BGMA with an LK-style iterated local search that lives in the same code base, uses the same language and runtime, and is timed on the same machine.

3 The algorithm

Given points $p_0, \ldots, p_{n-1}$ in the plane with distance matrix $D_{ij} = \|p_i - p_j\|_2$ (rounded to the nearest integer for TSPLIB `EUC_2D` instances, as the library prescribes), BGMA returns a Hamiltonian cycle. A tour is an array of city indices; for de-duplication it is put in a canonical form that starts at city 0 and takes the lexicographically smaller of the two directions. Preprocessing sorts, for every city, all other cities by distance ($O(n^2 \log n)$) and keeps a truncated candidate list of the $m = \max(10, \mathrm{round}(n/10))$ nearest.

BGMA runs one or more independent *units*. A unit (Algorithm 1) has five parts.

Beam islands. A beam state is a partial path with a start city, a current city, a visited set and a length. An island starts from two *poles*: the farthest pair of cities for the first island, and two seeded random cities for the others. Each pole seeds its own start state. At each of the $n - 1$ extension steps every state proposes at most four successors: the nearest unvisited neighbour of the current city, the second-nearest unvisited neighbour of the previous city, the third-nearest unvisited neighbour of the city before that, and the second-nearest unvisited neighbour of the current city. Successors are merged on a Zobrist hash of (visited set, last city, start city), keeping the shorter path. They are scored by $f = \mathrm{length} + D_{\mathrm{last,start}}$. The K best survive. The K completed tours (the *finals*) are all passed on. The beam is a cheap generator of diverse, structurally reasonable tours, in the spirit of filtered beam search [24]; it is not where the tour quality comes from.

Fast screen. Every beam final and every child is improved by a candidate-list local search: 2-opt [6] over the m-nearest lists with don't-look bits and a work queue [4], then Or-opt relocation of segments of length ≤ 3 [23] (both orientations, insertion next to a candidate neighbour), then 2-opt again. Iteration caps bound the work: at most $200n$ queue pops for 2-opt, and at most 20 accepted Or-opt moves, each followed by a rescan from the start of the tour. The cost is close to linear in n in practice, but the result is not in general a local optimum: on large instances the Or-opt cap binds, and the full polish below does the remaining work.

Elite pool and recombination. Screened tours are de-duplicated by canonical form, sorted by length and truncated to the best e. A generation creates c children. The first parent is drawn uniformly from the best $\min(4, |P|)$ tours, the second uniformly from the whole pool. The child is built by edge recombination (ERX) [27]: from a random start city, repeatedly move to the unvisited neighbour in the union of the parents' edges that has the fewest unvisited neighbours itself (ties: the shorter edge), and jump to the nearest unvisited city when no parental edge is available.

Algorithm 1 One BGMA unit. All random choices come from a generator seeded by (instance, seed).

$S \leftarrow$ finals of I beam islands (width K), each improved by the fast screen
$P \leftarrow$ best e distinct tours of S; polish the best q tours of P and merge them back
for $g = 1, \ldots, G$ **do**
  for $j = 1, \ldots, c$ **do**
    $a \leftarrow$ uniform from the best $\min(4, |P|)$ of P; $b \leftarrow$ uniform from P
    $x \leftarrow \mathrm{ERX}(a, b)$
    with probability `kickRate`: $x \leftarrow$ double-bridge(x)
    with probability `annealRate` (independently): $x \leftarrow$ anneal(x)
    $C_j \leftarrow$ fast-screen(x)
  polish the best $\min(q, c)$ of C; $P \leftarrow$ best e distinct tours of $P \cup C$
  if no improvement for 2 generations **then** merge one screened random-pole beam island into P
polish the best `finalPolish` tours of P and merge; **return** best tour in P

Perturbation. Two independent events may then modify the child. With probability `kickRate` it receives one double-bridge move [19]. Independently, with probability `annealRate`, it receives a short simulated anneal [16]. The anneal uses candidate-list 2-opt proposals and Metropolis acceptance. The temperature starts at $T_0 = \max(1, 0.02n)$ and is multiplied by 0.99998 per proposal. After $50n$ proposals without a new best it applies a double bridge and reheats to $\max(T, 0.4T_0)$. The anneal stops after $\min(A, \max(1000, 200n))$ proposals, where A is the level's cap, and returns the best tour it saw. T_0 is in absolute length units and is not normalised by the instance scale; we return to this in Section 7.

Selective full polish. After screening, the children are sorted and only the best $\min(q, c)$ receive the expensive local search: first-improvement 2-opt over all $O(n^2)$ pairs followed by Or-opt with segments up to L_p cities. The scan restarts after every accepted move and stops at a true local optimum for that neighbourhood. The same polish is applied to the best $\min(q, |P|)$ tours of the initial pool and, if `finalPolish` > 0, to that many pool tours at the end. Restricting the quadratic search to contenders is the main cost-control device: the fast screen is the filter, the full polish is the refinement.

Stagnation injection. If the best tour has not improved for two consecutive generations, one fresh beam island with random poles is run, its finals are screened and merged into the pool, and the counter resets.

Restarts and the effort dial. A level with R restarts runs R units with seed offsets and keeps the shortest tour. The five levels `low`, `mid`, `high`, `xhigh` and `max` (Table 1) differ only in budgets. The top level is a portfolio: `max` runs the `xhigh` configuration once and the `max` configuration with $R = 2$, i.e. three independent units, and returns the shortest of the three tours. By construction `max` is never worse than `xhigh` on the same instance and seed, and it costs the sum of its units. No other monotonicity between levels is guaranteed, and we observe violations (Section 5.2). Because the units share nothing, they can run on separate threads; the repository's worker pool does this and returns bit-identical results.

Table 1: The effort ladder (`src/hybrid/effort_profiles.js`). $r(x) = \max(1, \mathrm{round}(x))$. A and p_A are the anneal cap and probability, p_k the kick probability, L_p the polish Or-opt segment cap. `max` additionally runs one `xhigh` unit (portfolio). At most c children are polished per generation, so the cap q binds per generation only while $q < c$: from $n \approx 200$, 175 and 210 on, `high`, `xhigh` and `max` polish all c children.

level	I	K	e	G	c	q	final	R	A / p_A	p_k	L_p
`low`	2	$\max(12, r(n/4))$	$\max(8, r(n/20))$	$\max(2, r(n/50))$	4	2	0	1	8k / .25	.30	5
`mid`	4	$\max(16, r(n/3))$	$\max(16, r(n/10))$	$\max(4, r(n/25))$	10	6	2	1	20k / .35	.35	5
`high`	5	$\max(20, r(n/2.5))$	$\max(24, r(n/6))$	$\max(6, r(n/15))$	14	$\max(6, r(n/15))$	3	1	30k / .40	.40	6
`xhigh`	6	$\max(24, r(n/2))$	$\max(32, r(n/3))$	$\max(8, r(n/8))$	18	$\max(10, r(n/10))$	6	1	50k / .45	.45	8
`max`	8	$\max(32, r(n/2))$	$\max(48, r(n/2))$	$\max(10, r(n/8))$	18	$\max(10, r(n/12))$	8	2	60k / .50	.50	8

Cost. Per unit the work is roughly I beam islands (about $4nK$ node creations, each copying an n-bit visited set), $IK + Gc$ fast screens, $\min(q, e) + G\min(q, c)$ + final full polishes, and up to Gcp_A anneals of at most A proposals. The full polish dominates at large n. One scan is $O(n^2)$ and is restarted after every accepted move, and in the upper levels G grows linearly with n, while the polishes per generation are capped at c. The budgets are polynomial in n by construction, so BGMA is a polynomial-time heuristic and carries no optimality or approximation guarantee. The Euclidean TSP is NP-hard [25], and the result of any such heuristic says nothing about P versus NP.

The parameters were tuned in-sample. The ladder was set by one-factor sweeps on the same generator of uniform and clustered synthetic instances that Section 5.2 reports on. The TSPLIB instances in Section 5.3 were not used for tuning.

4 Experimental protocol

Implementation and machine. All code is dependency-free JavaScript, run on Node.js v22.23.2 on an Apple M3 Pro laptop (darwin, arm64). Times are wall clock for the solve, measured with `process.hrtime`. Instance generation and distance matrices are excluded. BGMA and the LK-style baseline build their own candidate lists, and in the synthetic benchmark that cost is included. On TSPLIB the BGMA unit times exclude instance construction (matrix and neighbour lists), while every LK-style run pays for its own matrix copy and candidate lists. That asymmetry favours BGMA in the equal-time comparison. Synthetic and runtime experiments are single-threaded. For `max` on TSPLIB, the three portfolio units run on three worker threads. For those runs we report the *serial cost*, the sum of the three unit times, which is what `max` costs on one core, and separately the wall clock with three workers. All randomness is seeded, so a run is a deterministic function of (instance, seed, level). Repeating a run returns the same tour; we test this in Section 5.4. The paper's data were produced by `paper/verify.sh` at commit `657ed2d80f3c`, and every table and number in it is generated from those files by `paper/gen_tables.js`.

Instances. *Synthetic:* uniform points in $[4, 96]^2$, and clustered points (3–4 Gaussian clusters, $\sigma = 8$, clipped to the same square), produced by a seeded generator. These are the families the ladder was tuned on. *TSPLIB:* the fourteen `EUC_2D` instances eil51, berlin52, st70, kroA100, pr124, bier127, ch130, tsp225, fl417, pcb442, u574, rat783, pr1002 and u1060, with their published optima [26]. The distances are rounded as the library specifies, so tour lengths are directly comparable with the

optima. *Robustness families:* points in convex position, collinear points, an integer grid, duplicated and coincident points, and uniform points scaled by 10^{-3} and 10^{6}.

Quality measures. We use three measures and keep them strictly apart. (i) The *true gap* $L/L^* - 1$ against a known optimum L^*: Held–Karp at $n \le 22$, published optima on TSPLIB, or a construction (convex position, collinear points). (ii) The *bound gap* $L/\mathrm{LB} - 1$ against the Held–Karp 1-tree bound after a fixed number of subgradient iterations. This is a valid *upper bound* on the true gap. (iii) The *relative excess* over the shortest tour any tested method found on the same instance. This measure is used only to compare methods with each other. Every reported tour is checked to be a permutation, and its length is recomputed from the distance matrix.

Baselines. All baselines are in the same repository and are timed with the same harness:

- *Greedy*: a nearest-neighbour tour from one pole.
- *Christofides* [5]: MST, then a minimum-weight perfect matching on the odd-degree vertices (Edmonds' blossom algorithm [7]), an Euler tour and shortcutting.
- *LK-style ILS*: an iterated Lin–Kernighan search written for this repository. It is *not* Helsgaun's LKH. It uses 8-nearest-neighbour candidate sets (not α-nearness), sequential LK chains of depth 3 (moves up to 5-opt) with gain pruning, don't-look bits, and double-bridge kicks with chained acceptance. The initial tour is the best of four nearest-neighbour starts. It runs $\max(40, \mathrm{round}(30000/n))$ kicks and stops early after 50 kicks without improvement. We use its default budget throughout. Its seed changes only the kicks and starts.
- *Beam + polish*: the beam of Section 3 with every final fully polished ($L_p = 5$), best of four restarts. This was the pipeline before BGMA, so it doubles as an ablation of the memetic stage.

Concorde and LKH were not run. Section 6 discusses what that omission means.

5 Results

5.1 Exact optima at $n \le 22$

On 26 synthetic instances (ten each at $n = 18$ and 20, six at $n = 22$; uniform and clustered), every effort level returned the Held–Karp optimum: 26/26 at each of the five levels. Held–Karp itself took 247 ms–289 ms per instance at $n = 20$ and 1.21 s–1.46 s at $n = 22$, against 1.3 ms (`low`) to 65 ms (`max`) for BGMA (median, $n = 20$). At these sizes all levels are equivalent, and the instances are too easy to separate them. This is a correctness check, not evidence of strength.

5.2 Synthetic instances, $n = 20$ to 400

Table 2 gives the median time and the mean excess over the best tour any method found, over four instances per size (two seeds, uniform and clustered). Figure 1 plots the same data at $n = 200$ and 400 as time against excess.

Five observations follow.

The dial works, at a steep price. Each level costs about 2–3.5 times the previous one. The cost ratios at $n = 400$ are 3.1, 3.3, 2.4, 3.2. Up to $n = 400$ each level also cuts the remaining excess roughly in half, from +2.22% at `low` to +0.00% at `max`. Fitted between $n = 20$ and 400, time grows as about $n^{2.1}$ at `low` and $n^{2.2}$ at `max`, but the growth steepens with n: between 100 and 400

Table 2: Synthetic instances: median solve time and mean relative excess over the best tour any method found on the same instance (4 instances per cell). Excess is a *relative* measure. At $n = 20$ every listed heuristic except greedy and Christofides is optimal (Section 5.1).

	median time					mean excess over best found				
method	$n = 20$	$n = 50$	$n = 100$	$n = 200$	$n = 400$	$n = 20$	$n = 50$	$n = 100$	$n = 200$	$n = 400$
Greedy nearest neighbour	3 µs	6 µs	13 µs	24 µs	50 µs	+14.03%	+27.72%	+31.90%	+31.05%	+26.46%
Christofides	220 µs	613 µs	1.1 ms	11 ms	27 ms	+10.50%	+10.07%	+13.07%	+13.49%	+13.03%
Beam + polish (4 restarts)	1.1 ms	5.9 ms	43 ms	441 ms	4.10 s	0.00%	+0.04%	+0.69%	+1.54%	+2.22%
LK-style ILS	13 ms	50 ms	97 ms	672 ms	1.40 s	0.00%	0.00%	0.00%	+0.11%	+0.37%
BGMA `low`	1.3 ms	4.1 ms	12 ms	83 ms	622 ms	0.00%	+0.12%	+0.52%	+1.70%	+2.22%
BGMA `mid`	5.0 ms	14 ms	37 ms	222 ms	1.95 s	0.00%	0.00%	+0.03%	+0.69%	+1.13%
BGMA `high`	9.6 ms	31 ms	84 ms	625 ms	6.37 s	0.00%	0.00%	+0.02%	+0.31%	+0.60%
BGMA `xhigh`	16 ms	52 ms	212 ms	1.81 s	15.0 s	0.00%	0.00%	0.00%	+0.02%	+0.12%
BGMA `max`	65 ms	203 ms	676 ms	5.46 s	48.6 s	0.00%	0.00%	0.00%	+0.01%	+0.00%
1-tree bound (400 it.)	96 µs	3.5 ms	15 ms	66 ms	263 ms					
Held–Karp	299 ms	—	—	—	—					

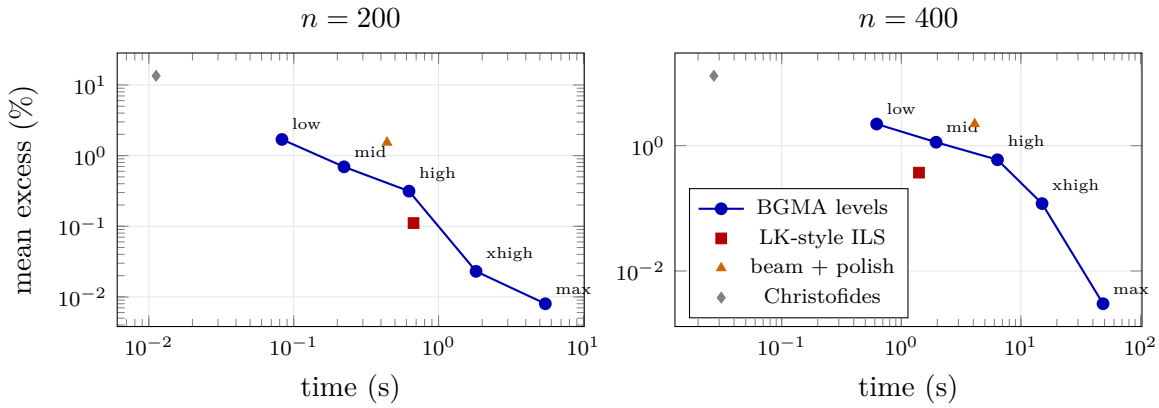


Figure 1: Time against mean relative excess on the synthetic instances (log–log; excess floored at 0.003% for plotting). BGMA levels are joined in order `low`→`max`.

the exponents are 2.9 and 3.1. This matches the cost model of Section 3, in which the number of quadratic polishes itself grows with n. The LK-style baseline grows as $n^{1.9}$ over the same range.

From $n = 200$ an LK-style ILS is the better trade-off below the top two levels. At $n = 200$ it takes about as long as `high` (672 ms against 625 ms) for about a third of the excess (+0.11% against +0.31%). At $n = 400$ the LK-style baseline takes 1.40 s for +0.37%. That beats `mid` (1.95 s, +1.13%) and `high` (6.37 s, +0.60%) on both time and quality. `low` is faster (622 ms) but its excess is much larger (+2.22%). Only `xhigh` and `max` find shorter tours (+0.12% and +0.00%), and they need 11 and 35 times the LK-style baseline's time. At $n \leq 100$ the picture reverses. Both methods are near-optimal, and BGMA's middle levels get there faster (at $n = 100$: `mid` 37 ms, LK-style 97 ms).

The memetic stage earns its place over its predecessor. The beam + polish pipeline reaches the same quality as `low` at $n = 400$ (+2.22% against +2.22%) but takes 6.6 times as long. Polishing every beam final is wasteful; polishing only the best screened candidates is not.

The guarantee is far from typical behaviour. Christofides runs in milliseconds and is +10.07% to +13.49% above the best found tour from $n = 50$ up. From $n = 50$ its excess is 6–86 times that of `low`.

Table 3: TSPLIB `EUC_2D` instances against published optima. Gaps are true gaps $L/L^* - 1$. "Serial time" is the median CPU time of one `max` run (sum of its three units); with three worker threads the wall clock is about a third of it. "Below opt." is how far the 600-iteration 1-tree bound lies under the optimum, i.e. how much a bound gap would overstate the true gap.

			BGMA `max`				LK-style, one run			LK-style, equal time			1-tree
instance	n	seeds	opt. hits	median gap	best gap	serial time	opt. hits	median gap	time	best gap	runs	reached median	below opt.
eil51	51	10	9/10	0.00%	0.00%	352 ms	1/10	+0.23%	19 ms	0.00%	26	44 ms	0.82%
berlin52	52	10	10/10	0.00%	0.00%	385 ms	10/10	0.00%	45 ms	0.00%	10	45 ms	0.00%
st70	70	10	10/10	0.00%	0.00%	496 ms	8/10	0.00%	77 ms	0.00%	7	136 ms	0.59%
kroA100	100	10	10/10	0.00%	0.00%	876 ms	10/10	0.00%	127 ms	0.00%	7	131 ms	1.62%
pr124	124	10	10/10	0.00%	0.00%	1.44 s	8/10	0.00%	331 ms	0.00%	5	324 ms	1.63%
bier127	127	10	10/10	0.00%	0.00%	1.58 s	3/10	+0.83%	224 ms	0.00%	6	636 ms	0.72%
ch130	130	10	10/10	0.00%	0.00%	1.68 s	3/10	+0.29%	429 ms	0.00%	5	1.10 s	0.56%
tsp225	225	10	10/10	0.00%	0.00%	7.75 s	3/10	+0.37%	451 ms	0.00%	15	1.44 s	0.97%
fl417	417	10	10/10	0.00%	0.00%	57.6 s	1/10	+0.78%	3.05 s	0.00%	20	20.3 s	5.34%
pcb442	442	10	0/10	+0.66%	+0.51%	64.8 s	0/10	+0.56%	1.13 s	+0.25%	60	1.18 s	0.56%
u574	574	3	0/3	+0.50%	+0.33%	3.5 min	0/3	+1.14%	3.23 s	+0.47%	62	3.2 min	0.53%
rat783	783	3	0/3	+1.15%	+1.04%	9.7 min	0/3	+1.23%	1.74 s	+0.69%	298	5.61 s	0.38%
pr1002	1002	1	0/1	+2.10%	+2.10%	39.8 min	0/1	+2.19%	5.73 s	+1.07%	400	12.1 s	0.90%
u1060	1060	1	0/1	+1.45%	+1.45%	58.4 min	0/1	+2.06%	6.66 s	+0.55%	555	12.8 s	0.86%

Monotonicity across levels is not guaranteed. Only the `max` $\geq$ `xhigh` step is monotone by construction (Section 3). On individual instances a cheaper level sometimes beats a more expensive one; on rat783, for example, the `xhigh` unit beats both `max` restarts (Section 5.3).

5.3 TSPLIB: true gaps against published optima

Table 3 scores BGMA `max` and the LK-style baseline against the published optimum of each instance. Distances are rounded as `EUC_2D` prescribes. Instances up to $n = 442$ were run with ten seeds. The larger ones were run with one to three seeds, because one `max` run there costs minutes to over an hour of CPU time. The *equal-time* columns restart the LK-style search with new seeds until it has spent the median serial cost of one BGMA `max` run on that instance. They report the best tour found within that budget, the number of restarts it allowed, and the cumulative CPU time at which the restarts first reached a tour at least as short as BGMA's median.

Up to $n = 442$. BGMA `max` returned the optimum in 89 of 100 runs. It did so on all ten seeds for 8 of the 10 instances and on nine seeds for eil51. It never did on pcb442 (median +0.656%, best +0.514%). Single LK-style runs are much less reliable: 47 of 100 were optimal. They are also much cheaper, and the best of ten LK-style runs reaches the optimum on 9 of the 10 instances, the same ones as BGMA. On fl417 the ten LK-style runs together cost 29.0 s, against 57.6 s for one `max` run. On pcb442 the LK-style baseline is better outright: a median of +0.560% in 1.13 s per run, against +0.656% for `max` in 64.8 s. Instance structure matters as much as size. fl417, with its dense clusters, is solved on every seed, while pcb442, which is only slightly larger, is solved on none.

Beyond $n = 500$. The median gaps of `max` range from +0.50% to +2.10%. These are true gaps: the excess is genuine tour excess, not slack in a bound. The three portfolio units show the top of the dial flattening. On u574 the `xhigh` unit reached +0.45% and the two `max` restarts +0.33% and +0.53%. On rat783 they reached +1.04% against +1.89% and +1.52%. On pr1002 they reached +2.16% against +2.10% and +2.26%, and on u1060 +1.96% against +1.90% and +1.45%. The heavier `max` configuration is not reliably better than `xhigh` on the same instance. Its contribution

Table 4: Degenerate and known-optimum families. "Gap to bound" is listed per n (20, 100, 400) and is a ceiling on the true gap. *fine* and *coarse* are the same point set scaled by 10^{-3} and 10^{6}.

family	level	n	runs	invalid	result
convex	`low`	20, 100, 400	3	0	hull cycle (optimal) at every n
convex	`max`	20, 100, 400	3	0	hull cycle (optimal) at every n
line	`low`	20, 100, 400	3	0	known optimum at every n
line	`max`	20, 100, 400	3	0	known optimum at every n
grid	`low`	20, 100, 400	3	0	gap to bound 0.00%, 0.00%, +0.41%
grid	`max`	20, 100, 400	3	0	gap to bound 0.00%, 0.00%, 0.00%
dup	`low`	20, 100, 400	3	0	gap to bound 0.00%, +1.21%, +2.32%
dup	`max`	20, 100, 400	3	0	gap to bound 0.00%, +0.50%, +0.85%
same	`low`	20, 100, 400	3	0	known optimum at every n
same	`max`	20, 100, 400	3	0	known optimum at every n
fine	`low`	20, 100, 400	3	0	gap to bound 0.00%, +0.95%, +3.84%
fine	`max`	20, 100, 400	3	0	gap to bound 0.00%, +0.94%, +1.65%
coarse	`low`	20, 100, 400	3	0	gap to bound 0.00%, +0.95%, +3.96%
coarse	`max`	20, 100, 400	3	0	gap to bound 0.00%, +0.94%, +1.65%

is mostly that of extra independent samples. A single `max` run on pr1002 costs 39.8 min of CPU time (13.6 min on three threads).

At equal CPU time. The comparison that matters is the one at equal cost, and it is unfavourable to BGMA. Restarted for the CPU time of one `max` run, the LK-style search matched BGMA's median tour on 9 instances and beat it on 5 (pcb442, u574, rat783, pr1002, u1060). It was never worse. On the 10 instances up to $n = 442$ it found the optimum on 9, the same instances BGMA solves. On pcb442 it reached +0.250% against BGMA's +0.656%. On the large instances the difference is a matter of orders of magnitude, not percentages. The LK-style restarts reached BGMA's median pr1002 tour after 12.1 s of CPU time, where BGMA needs 39.8 min. On u1060 they reached it after 12.8 s (BGMA: 58.4 min), and they ended at +1.074% and +0.547% against BGMA's +2.095% and +1.451%. The reliability of `max` on small instances is therefore not an advantage per unit of CPU time: a simple multi-start LK search buys the same reliability more cheaply.

The bound gap misleads. The last column of Table 3 shows the 1-tree bound 0.00–5.34% below the optimum, depending on the instance. An optimal fl417 tour has a bound gap of +5.639%, while an optimal berlin52 tour has 0.000%. Had we reported bound gaps, as is common when no optimum is known, fl417 would have looked like BGMA's worst instance. It is one of its best.

5.4 Robustness, determinism and degenerate inputs

The stress harness ran `low` and `max` on seven families at $n = 20$, 100 and 400 (42 runs, Table 4). Every tour was checked to be a valid permutation with a correctly recomputed length. No run produced an invalid tour (0 failures), and no tour came out shorter than the proven bound (0 violations).

Known optima by construction. For points in convex position the hull cycle is optimal, and both levels returned it at every size. At $n = 1000$, `low` returned the hull cycle in 17.2 s and `max` returned the hull cycle in 15.0 min of CPU time. Collinear points (optimum: twice the span) and

coincident points (optimum: 0) were solved exactly. On the 20×20 grid, `max` met the 1-tree bound, which proves its tour optimal, while `low` did not.

The bound can be very loose. On collinear points the tour is optimal by construction, yet its bound gap reads +36.81% at $n = 100$ and +69.23% at $n = 400$. A degree-constrained relaxation cannot express the structure of a line. This is the same phenomenon as fl417 in Section 5.3, in a more extreme form.

Scale dependence. The *fine* and *coarse* families are one point set at two scales, so a scale-free method would return the same relative gap on both. `max` nearly does (+1.65% against +1.65% at $n = 400$). `low` does not (+3.84% against +3.96%), because the anneal temperature is set in absolute units (Section 7).

Determinism. Every level was run three times on each of two $n = 100$ instances (30 runs in 10 groups). In 10 of the 10 groups the repeats returned the same length and the same tour. Runs are a deterministic function of (instance, seed, level), also under the worker pool.

6 Discussion

What the evidence supports. BGMA is exact on every instance small enough for Held–Karp. At `max` it returns the published optimum on 9 of the 10 TSPLIB instances up to $n = 442$, on nearly every seed, and on none of the ten seeds of pcb442 (Table 3). On the synthetic families its effort dial trades time for quality in a regular way up to a few hundred cities. Beyond about 500 cities the residual is real tour excess of +0.50% to +2.10% (median over seeds), not an artefact of the bound, and the top of the ladder stops paying.

What it does not support. BGMA is not competitive with the state of the art, or even with a simple baseline. The clearest evidence is in this paper itself. An LK-style iterated local search of a few hundred lines, in the same language, beats BGMA's lower levels on synthetic instances from $n = 200$ (Section 5.2). Restarted for the CPU time of one `max` run, it matches or beats `max` on every TSPLIB instance, and on the largest ones it is faster by orders of magnitude (Section 5.3). The likely reason is that LK moves (up to 5-opt) reach much deeper local optima than BGMA's 2-opt/Or-opt polish, so each restart is worth more than a generation of recombination. Real LK implementations are stronger still: LKH [12] and EAX [22] find optimal or near-optimal tours on instances of these sizes in seconds, and Concorde [1] proves optimality for all of them. We did not run those codes, so we make no quantitative claim about them. A reader who needs good TSP tours should use them. What BGMA adds is a small, readable, dependency-free implementation with an explicit budget dial and a measurement record that can be reproduced end to end.

Why the top of the ladder stops paying. For $n \geq 80$, `xhigh` and `max` use the same beam width ($n/2$) and the same number of generations ($n/8$). What `max` adds is more islands, a larger pool, more final polishing and a second restart, and it pays for all of it in full. Beyond $n \approx 210$ both levels polish all $c = 18$ children of every generation, so their polishing work per generation is identical. `max` polishes *fewer* tours of the initial pool ($r(n/12)$ against $r(n/10)$), and its second parent is drawn from a pool of $n/2$ tours rather than $n/3$, so it is more often a weak tour. Either may be why the `xhigh` unit wins on rat783; we have not tested which. With ERX, a 2-opt/Or-opt polish and weak perturbation, more of the same search mostly revisits the same basins. A stronger

local search (Or-3opt, LK) inside the memetic loop, as the equal-time results also suggest, or a recombination that preserves more structure (EAX), is the likely lever, not more budget.

The 1-tree bound is a poor yardstick for heuristics. The bound gap over-states the true gap by an amount that depends on the instance, not on the tour. On fl417 an optimal tour reads as +5.639% above the bound, on berlin52 as 0.000%, and on collinear points an optimal tour reads +69.23% above the bound (Section 5.4). Two heuristics compared through bound gaps on different instances are therefore not compared at all. Wherever a true optimum is available it should be used.

7 Limitations and threats to validity

- *In-sample tuning.* The ladder was tuned on the synthetic families of Section 5.2. TSPLIB was held out, but there are only fourteen instances, chosen as standard `EUC_2D` benchmarks of increasing size and not at random.
- *Seeds.* TSPLIB instances up to $n = 442$ use ten seeds. Larger instances use one to three, because a single `max` run there costs 3.5 min to 58.4 min of CPU time. Their gaps are a few observations, not distributions.
- *One machine, one runtime.* All times come from one laptop and one Node.js version. Ratios between methods in the same harness are more portable than absolute times.
- *Baselines.* The LK-style baseline is ours and simpler than LKH. On TSPLIB it is compared both per run and at equal CPU time. On the synthetic instances it runs at its default budget only, so those comparisons are not time-matched.
- *Scale dependence.* The anneal temperature $T_0 = \max(1, 0.02n)$ is in absolute distance units. On instances with very large or very small coordinates the anneal degenerates into greedy descent or a random walk (it returns the best tour it saw, so it cannot make a child worse). The other operators are scale-free. Normalising T_0 by the mean candidate edge length is the obvious fix and has not been done.
- *Synthetic optima.* Above $n = 22$ the synthetic families have no known optimum. For them we report only relative excess and the bound gap, which is a ceiling.

8 Conclusion

BGMA is a compact memetic heuristic with an explicit effort dial. Measured against known optima it is exact on small instances, reliably optimal at its top level on most TSPLIB instances up to a few hundred cities, and +0.50% to +2.10% above the optimum (median over seeds) from 574 to 1060 cities. The dial is regular up to 400 cities and flattens beyond. The same measurements show its limits plainly. An LK-style iterated local search of comparable size, restarted for the same CPU time, matches or beats BGMA's top level on every TSPLIB instance tested. Mature solvers are in a different class. We release the implementation, the harnesses and every data file behind this paper, so that each number can be regenerated with one script. The most promising change is to replace the 2-opt/Or-opt polish with LK moves, i.e. to make BGMA a memetic algorithm over LK local optima in the manner of Merz and Freisleben [20], and to normalise the annealing scale. Whether the memetic layer then adds anything over plain restarts is the open question.

Reproducibility

Code, data and this paper's sources are at `https://github.com/Mhmdrza/polynomialTSP` (MIT licence). `paper/verify.sh` reruns every experiment; TSPLIB files must be obtained separately. `paper/gen_tables.js` regenerates every table and number from `paper/data/`, and Appendix A lists the commands. Each TSPLIB result file stores the tour, so the reported lengths can be checked independently.

Acknowledgements

Parts of the implementation, the experiment harnesses and the text were developed with the help of an AI coding assistant (Anthropic's Claude). All experiments were run by the author, and the author is responsible for the content.

References

[1] David L. Applegate, Robert E. Bixby, Vašek Chvátal, and William J. Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006.

[2] Sanjeev Arora. Polynomial time approximation schemes for Euclidean traveling salesman and other geometric problems. *Journal of the ACM*, 45(5):753–782, 1998. doi: 10.1145/290179.290180.

[3] Richard Bellman. Dynamic programming treatment of the travelling salesman problem. *Journal of the ACM*, 9(1):61–63, 1962. doi: 10.1145/321105.321111.

[4] Jon Louis Bentley. Fast algorithms for geometric traveling salesman problems. *ORSA Journal on Computing*, 4(4):387–411, 1992. doi: 10.1287/ijoc.4.4.387.

[5] Nicos Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. Technical Report 388, Graduate School of Industrial Administration, Carnegie Mellon University, 1976.

[6] G. A. Croes. A method for solving traveling-salesman problems. *Operations Research*, 6(6):791–812, 1958. doi: 10.1287/opre.6.6.791.

[7] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965. doi: 10.4153/CJM-1965-045-4.

[8] Michael Held and Richard M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962. doi: 10.1137/0110015.

[9] Michael Held and Richard M. Karp. The traveling-salesman problem and minimum spanning trees. *Operations Research*, 18(6):1138–1162, 1970. doi: 10.1287/opre.18.6.1138.

[10] Michael Held and Richard M. Karp. The traveling-salesman problem and minimum spanning trees: Part II. *Mathematical Programming*, 1(1):6–25, 1971. doi: 10.1007/BF01584070.

[11] Keld Helsgaun. An effective implementation of the Lin–Kernighan traveling salesman heuristic. *European Journal of Operational Research*, 126(1):106–130, 2000. doi: 10.1016/S0377-2217(99)00284-2.

[12] Keld Helsgaun. General k-opt submoves for the Lin–Kernighan TSP heuristic. *Mathematical Programming Computation*, 1(2–3):119–163, 2009. doi: 10.1007/s12532-009-0004-6.

[13] Keld Helsgaun. An extension of the Lin–Kernighan–Helsgaun TSP solver for constrained traveling salesman and vehicle routing problems. Technical report, Roskilde University, 2017.

[14] David S. Johnson and Lyle A. McGeoch. The traveling salesman problem: A case study in local optimization. In Emile H. L. Aarts and Jan Karel Lenstra, editors, *Local Search in Combinatorial Optimization*, pages 215–310. John Wiley and Sons, 1997.

[15] David S. Johnson and Lyle A. McGeoch. Experimental analysis of heuristics for the STSP. In Gregory Gutin and Abraham P. Punnen, editors, *The Traveling Salesman Problem and Its Variations*, pages 369–443. Kluwer Academic Publishers, 2002.

[16] Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.

[17] Shen Lin and Brian W. Kernighan. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2):498–516, 1973. doi: 10.1287/opre.21.2.498.

[18] Helena R. Lourenço, Olivier C. Martin, and Thomas Stützle. Iterated local search. In Fred Glover and Gary A. Kochenberger, editors, *Handbook of Metaheuristics*, pages 320–353. Kluwer Academic Publishers, 2003.

[19] Olivier Martin, Steve W. Otto, and Edward W. Felten. Large-step Markov chains for the traveling salesman problem. *Complex Systems*, 5(3):299–326, 1991.

[20] Peter Merz and Bernd Freisleben. Memetic algorithms for the traveling salesman problem. *Complex Systems*, 13(4):297–345, 2001.

[21] Pablo Moscato. On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms. Caltech Concurrent Computation Program, C3P Report 826, California Institute of Technology, 1989.

[22] Yuichi Nagata and Shigenobu Kobayashi. A powerful genetic algorithm using edge assembly crossover for the traveling salesman problem. *INFORMS Journal on Computing*, 25(2):346–363, 2013. doi: 10.1287/ijoc.1120.0506.

[23] Ilhan Or. *Traveling Salesman-Type Combinatorial Problems and Their Relation to the Logistics of Regional Blood Banking*. PhD thesis, Northwestern University, 1976.

[24] Peng Si Ow and Thomas E. Morton. Filtered beam search in scheduling. *International Journal of Production Research*, 26(1):35–62, 1988. doi: 10.1080/00207548808947840.

[25] Christos H. Papadimitriou. The Euclidean travelling salesman problem is NP-complete. *Theoretical Computer Science*, 4(3):237–244, 1977. doi: 10.1016/0304-3975(77)90012-3.

[26] Gerhard Reinelt. TSPLIB—a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384, 1991. doi: 10.1287/ijoc.3.4.376.

[27] Darrell Whitley, Timothy Starkweather, and D'Ann Fuquay. Scheduling problems and traveling salesmen: The genetic edge recombination operator. In J. David Schaffer, editor, *Proceedings of the Third International Conference on Genetic Algorithms*, pages 133–140. Morgan Kaufmann, 1989.

[28] Shlomo Zilberstein. Using anytime algorithms in intelligent systems. *AI Magazine*, 17(3): 73–83, 1996.

A Reproduction

With Node.js ≥ 18 and no other dependency:

```
npm test                                  # self-tests
TSPLIB=/path/to/tsplib sh paper/verify.sh # all experiments
node paper/gen_tables.js                  # data -> tables
```

The steps of `verify.sh` can be run separately (`sh paper/verify.sh C` runs only the ten-seed TSPLIB step). Individual runs:

```
node tuning/validate_effort.js --ns=18,20 --seeds=5
node bench/runtime_bench.js --json=runtime.json --md=1
node tuning/tsplib_bench.js --file=pcb442.tsp --opt=50778 \
     --levels=lk,max --seed=1 --jobs=3
node tuning/stress_parallel.js --ns=400 --families=convex,line,grid --levels=low,max
node src/hybrid/hybrid_effort.js --n=200 --effort=max --bound=1
```

Runs are deterministic given (instance, seed, level): repeating a command reproduces the tour exactly, and only the times vary.